



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Scienze Matematiche, Fisiche e Naturali
Laurea Magistrale in Data Science, Calcolo Scientifico &
Intelligenza Artificiale

Master Thesis

THESIS' TITLE

NAME SURNAME

Advisor: *prof. Name Surname*

Co-Advisor: *prof. Name Surname*

Academic Year 2023-2024

ABSTRACT

Nowadays the huge amount of data generated by digital financial systems, together with the evolution of fraud schemes, make fraud detection a challenging and critical problem. Since illicit actors employ sophisticated strategies such as fraud rings and cyclic transfers to obscure their trails, traditional heuristic and rule-based detection systems frequently fail to provide an effective solution.

The primary goal of this thesis is to develop a detailed Python implementation of a Quantum Topological Graph Neural Network (QTGNN) [2]. This model combines graph neural networks, quantum state embedding and topological data analysis to capture higher-order, long-range dependencies and robust topological invariants that are typical in fraud schemes. Leveraging these features we will also be able to enhance the interpretability of the model. This is extremely important since, in regard to using deep learning models in the real world and particularly in legal settings, the model must be able to explain its decisions [4].

The model will be evaluated on four different transaction datasets: the Elliptic Bitcoin Dataset, Ethereum Phishing Transaction Networks, IBM AMLSim, and PaySim. To balance computational feasibility with practical applicability, the model will be trained on a quantum simulator. Only the final testing phase will be executed on an actual NISQ quantum computer.

CONTENTS

1	Introduction	9
1.1	The Growing Challenge	9
1.1.1	The Limitations of the Lonely Scammer	9
1.1.2	Fraud Rings	10
1.2	Proposed Solution: QTGNN	11
1.3	Thesis Objectives	12
2	Model description	13
2.1	Generalized Amplitude Encoding	13
2.2	Variational Quantum Graph Convolution	16
2.3	Topological Quantum Signature Extraction	19
2.4	Quantum-Classical Hybrid Anomaly Learning	20
2.5	Decision and Interpretability	22
A	A previous implementation of S_{normal}	25

LIST OF FIGURES

"Citation"
— *citation's author*

INTRODUCTION

Fraud detection has become a foundational pillar of modern economic security. The rapid digitization of financial services has opened new possibilities for fraudulent activities. Undetected financial frauds lead to massive economic losses for individuals and institutions, undermine public trust in the financial system and frequently serve to finance criminal organizations (e.g. money laundering). Consequently, developing robust mechanisms to identify and stop fraudulent transactions is critically important to preserving both market integrity and social stability.

1.1 THE GROWING CHALLENGE

Detecting fraud is becoming increasingly difficult. The sheer volume and velocity of digital transactions make manual oversight impossible and heavily strain classical machine learning architectures. Furthermore, the rise of decentralized finance (DeFi), Web3, and cryptocurrencies has introduced a paradigm of pseudo-anonymity that complicates traditional tracing. On top of that, the sophistication of the techniques used by scammers is constantly evolving.

1.1.1 *The Limitations of the Lonely Scammer*

Individual scammers typically rely on simple and intuitive techniques to introduce illicit funds into the legitimate economy. One of the most prevalent methods is known as “smurfing”. In simple terms, smurfing involves breaking down a large sum of illicit money into many smaller, inconspicuous chunks to deposit them without raising suspicion.

More formally, individual scammers rely on smurfing to avoid Anti-Money Laundering (AML) detection mechanisms, which usually flag transactions above a specific threshold $\tilde{x}$. By splitting their initial illicit

funds x into smaller transactions x_n , such that each transaction satisfies $x_n \leq \tilde{x}$, they minimize the risk of detection [1]. The mathematical expectation of profit π_s for a scammer engaging in smurfing can be defined as:

$$\pi_s = x(1 - v) \quad (1.1)$$

where x represents the total initial illicit funds, and $v \in (0, 1)$ is the friction cost or commission proportion associated with the smurfing process (e.g., the fee paid to mules).

While smurfing is effective for small amounts, its scalability is severely limited. When attempting to launder much larger volumes of funds, an individual scammer operating from a single account faces a significantly higher probability of detection $(1 - p)$, where p is the probability of successfully avoiding detection. If caught, the scammer is subjected to a severe monetary penalty T , which includes both the total confiscation of the funds and associated judicial fines [1]. Therefore, operating as a “lonely scammer” becomes mathematically and practically unviable for large-scale operations due to the massive expected cost of $(1 - p)T$.

1.1.2 *Fraud Rings*

In regard to overcome those issues, criminals have improved their techniques by employing highly sophisticated obfuscation tactics. Instead of acting alone, they form complex “fraud rings” to distribute illicit funds across vast networks of temporary accounts. To further break the traceable link between sender and receiver, illicit actors frequently use cyclic transfers and cryptocurrency **mixers** services that pool together funds from multiple users, mix them, and redistribute them at random intervals and amounts. These advanced tactics create dense, highly camouflaged transaction topologies that easily evade threshold-based rules and simple heuristic detection methods.

To achieve their goal they utilize a strategy of transaction layering, often characterized by a topological depth L , and network diversification to obscure the money trail [1, 6]. By spreading the funds through a complex topology, the fraud ring successfully increases its non-detection probability p_L , which now depends explicitly on the layering depth L . The expected profit of a fraud ring utilizing L layers can be mathematically expressed as:

$$E[\pi_L] = p_L x - (1 - p_L)T - c(L) \quad (1.2)$$

where $E[\pi_L]$ is the expected profit, p_L is the enhanced probability of evading detection when L layers are used, x is the total illicit funds, T is the total penalty if caught, and $c(L)$ represents the monotonic cost function associated with establishing and maintaining the L transaction layers. Because the fraud ring dramatically increases the probability of evading detection ($p_L \rightarrow 1$ as L increases), the expected payoff remains highly profitable even when factoring in the complex layering costs $c(L)$ [1].

1.2 PROPOSED SOLUTION: QTGNN

Traditional Graph Neural Networks (GNNs) have shown promise in fraud detection by utilizing local message-passing mechanisms. However, they often struggle to capture the long-range dependencies and higher-order topological structures that characterize modern, sophisticated fraud rings.

To address these limitations, we propose the use of a Quantum Topological Graph Neural Network (QTGNN). By combining the principles of Quantum Computing, Graph Theory, and Topological Data Analysis (TDA), the QTGNN offers a remarkably robust solution. It leverages quantum embeddings to capture multiple complex transaction pathways simultaneously and employs topological invariants (such as persistent homology) to identify the underlying structural “shape” of the fraud network, isolating it from deliberate noise and obfuscation [2]. By doing so we aim to address three fundamental challenges:

- **Leveraging quantum embeddings for richer relational representation:** Traditional representations struggle to model the high-dimensional pathways of fraud rings. Quantum embeddings encode this complex relational information by exploiting quantum superposition and entanglement. This allows the model to represent multiple transaction pathways and interdependencies simultaneously, capturing subtle patterns distributed across the network that classical local message-passing GNNs often overlook [2].
- **Incorporating topological invariants for robustness and interpretability:** Topological Data Analysis (TDA) has gained significant attention for its ability to extract invariant features that characterize the global structure of data [7]. By extracting topological descriptors such as persistent homology, the QTGNN captures global structural features that remain invariant to noise and minor perturbations.

This not only makes the representations robust against the spurious connections generated by scammers but also enhances the model’s interpretability by identifying the higher-order motifs driving the anomaly detection.

- **Ensuring scalability and practicality on NISQ devices:** While quantum models offer theoretical advantages, their practical deployment is heavily constrained by the limitations of Noisy Intermediate-Scale Quantum (NISQ) hardware [11]. The QTGNN framework addresses these bottlenecks through a NISQ-aware design that employs circuit optimization, error mitigation, and hybrid quantum-classical computation strategies. This ensures that the quantum-enhanced graph learning remains feasible, scalable, and fully deployable on real-world financial transaction networks [2].

1.3 THESIS OBJECTIVES

The primary objective of this thesis is to develop a comprehensive Python implementation of the QTGNN specifically tailored for fraud detection.

To evaluate the model’s performance against varying structural challenges, the implementation will be tested on several distinct datasets:

- **The Elliptic Bitcoin Dataset:** A real-world dataset mapping Bitcoin transaction flows that contains rich topological structures obfuscated by illicit actors.
- **Ethereum Phishing Transaction Networks:** A real-world dataset where accounts and transactions are modeled as nodes and edges to perform phishing account classification.
- **IBM AMLSim:** A dataset containing synthetic everyday transactions embedded with specific alert patterns and anomalies that mimic actual money laundering typologies, such as smurfing.
- **PaySim:** A synthetic financial dataset for evaluating broader classification capabilities.

Given the current limitations of quantum hardware, this research adopts a hybrid testing strategy. The QTGNN model will be trained mainly using a quantum simulator, while the final evaluation and testing will be performed on an actual quantum computer. This allow to significantly reduce computational cost whilst validate the model’s viability on Noisy Intermediate-Scale Quantum (NISQ) devices.

MODEL DESCRIPTION

During the years the effectiveness of graph-based models in fraud detection brought them to be the standard approach in the field [9]. For our project we will consider a financial transaction graph $G = (V, E)$, $|V| \doteq N$, where each node $v \in V$ represents an account and each edge $(u, v) \in E$ represents a transactions between accounts u and v . Each edge is associated with a weight $W(u, v) = w_{uv}$ defined as

$$w_{uv} \doteq \sum_{t \in T_{uv}} \text{amount}(t) \quad (2.1)$$

where T_{uv} is the set of transactions between accounts u and v . Node centrality is also taken in consideration and it's computed as

$$\alpha_i \doteq \frac{\deg(i)}{\sum_{j \in V} \deg(j)} \quad (2.2)$$

Before moving forward the graph is filtered to remove edges with low weights

$$E \leftarrow \{(u, v) \in E \mid w_{uv} > \tau_{\text{filter}}\} \quad (2.3)$$

This is true for every dataset mentioned in Section 1.3 but for the Bitcoin transaction graph where nodes are transactions and edges are the flow of Bitcoin.

2.1 GENERALIZED AMPLITUDE ENCODING

Generalized Amplitude Encoding with Entanglement Seeding (GAES) is performed to encode the graph structure and features into quantum states. The idea behind standard amplitude encoding is to represent data

as amplitudes of a quantum state. Given a vector $v \in \mathbb{R}^N$, $N = 2^n$ with $\|v\| = 1$, the corresponding quantum state is

$$|\psi\rangle = \sum_{i=0}^{2^n-1} v_i |i\rangle \quad (2.4)$$

where ψ is a quantum state with n qubits.

In our case we want to encode not only node features but also graph structure. We achieve this by using entanglement seeding [2], which introduces entanglement between qubits to capture relationships between nodes.

$$|\psi_G\rangle = \frac{1}{\sqrt{Z}} \sum_{i,j} A_{ij} \mathcal{E}_{ij}(\theta_e) |i\rangle |j\rangle \quad (2.5)$$

where

$$A_{ij} \doteq \begin{cases} w_{ij} & \text{if } i \neq j \\ \alpha_i & \text{if } i = j \end{cases} \quad (2.6)$$

$$Z \doteq \sum_{i,j} |A_{ij}|^2 \quad (2.7)$$

and

$$\mathcal{E}_{ij}(\theta_e) \doteq \exp(-i\theta_e \sigma_i^x \sigma_j^x) \quad (2.8)$$

is a quantum gate that applies entanglement between qubits i and j , σ_i^A with $A \in \{X, Y, Z\}$ are the Pauli operators acting on qubit i and θ_e are parameters that will be optimized during training so that the state physically embodies the interactions defined by $H(G)$.

This definition is, however, subject to a big dimensionality flaw. Under our amplitude encoding, the quantum state lives in an $2n$ -qubit Hilbert space where $n = \log_2(N)$. The operator σ_i^x is a global Pauli-X gate acting on physical qubit i , yet here the index i ranges over node indices $[0, N - 1]$, which is dimensionally inconsistent with the n -qubit register. More critically, even if the index were in range, applying σ^x based on a binary mapping would universally flip bits across the entire superposition, completely scrambling the graph topology instead of selectively linking node i to node j .

My solution is to replace the entanglement operator with a *subspace projector* (exchange operator) that acts exclusively on the subspace spanned by the computational basis states $|i\rangle$ and $|j\rangle$:

$$\mathcal{E}_{ij}(\theta_e) \doteq \exp(-i\theta_e(|i\rangle\langle j| + |j\rangle\langle i|)) \quad (2.9)$$

The generator $(|i\rangle\langle j| + |j\rangle\langle i|)$ is the generator of a continuous-time quantum walk between nodes i and j . When exponentiated, it applies an $SU(2)$ rotation strictly to the amplitudes of those two connected nodes, leaving all other amplitudes untouched. This is consistent with the amplitude encoding paradigm and achieves the authors' intended goal of edge-specific entanglement.

$H(G)$ is the graph Hamiltonian and it includes higher-order interactions to capture multi-party transactions. The original paper define it as

$$H(G) \doteq \sum_{(i,j) \in E} w_{ij}(\sigma_i^x \sigma_j^x + \sigma_i^y \sigma_j^y + \sigma_i^z \sigma_j^z) + \sum_{i \in V} \alpha_i \sigma_i^z + \sum_{(i,j,k) \in \Delta} \gamma_{ijk} \sigma_i^z \sigma_j^z \sigma_k^z \quad (2.10)$$

where γ_{ijk} captures the three-way interactions between nodes i , j , and k . The original paper doesn't provide a specific definition for γ_{ijk} , just the description of its purpose. A possible interpretation is the geometric mean of the weights of the edges forming the triangle.

$$\gamma_{ijk} \doteq \sqrt[3]{w_{ij} \cdot w_{jk} \cdot w_{ki}} \quad (2.11)$$

From my prespective there is a big flaw in the definition of the Hamiltonian. As before, from equation 2.10 we can see that the Paoli gates are being applied to the i -th qubits, this index i layes in the range $[0, N - 1]$ where N is the number of nodes in the graph. But from equation 2.5 we can see that after the encoding we will have $2n$ qubits, where $n = \log_2(N)$. This is a contradiction and obviously we cannot afford to have N qubits since in real world applications N can be in the order of millions.

My solution is to redefine the Hamiltonian using lernable parameters that will capture the interactions between nodes without explicitly mapping them to qubits [3]. The new Hamiltonian becomes

$$H \doteq H_{\text{eff}}(\Theta) = \underbrace{\sum_{q=0}^{n-1} \phi_q \sigma_q^z}_{\text{Global Bias}} + \underbrace{\sum_{q < r}^{n-1} \theta_{qr} \sigma_q^z \sigma_r^z}_{\text{Qubit Correlations}} + \underbrace{\sum_{q < r < s}^{n-1} \psi_{qrs} \sigma_q^z \sigma_r^z \sigma_s^z}_{\text{Higher-Order Entanglement}} \quad (2.12)$$

This consists in a parameterized diagonal Hamiltonian that strictly operates on the correct $2n$ -qubit register, making the Variational Quantum Circuit (VQC) mathematically executable. This happens at the expense of the loss of explicitly topology. In the original, albeit flawed concept, the Hamiltonian explicitly used edge weights w_{ij} to drive the quantum interactions. With this new implementation we are assuming that the information contained in the quantum state ψ_G is sufficient to reconstruct the graph properties and that the parameters Θ will learn the topological information from the data.

After the encoding we will end up with a quantum state ψ_G composed by $2n$ qubits, where $n \doteq \log_2(N)$. Usually for amplitude encoding we would need only n qubits, the other n are defined as *environment* and are introduced to perform the entanglement seeding. In the equation 2.5 those correspond to the second register $|j\rangle$.

$$|\psi_G\rangle \in \mathcal{H}_{\text{source}} \otimes \mathcal{H}_{\text{env}} \quad (2.13)$$

Since we are interested only in the source register, we will trace out the environment qubits to obtain a reduced density matrix ρ_G . The use of the density matrix allows to represent mixed states and capture statistical properties of the quantum system.

$$\rho_{\text{total}} = |\psi_G\rangle\langle\psi_G| \quad (2.14)$$

$$\rho_G = \text{Tr}_{\text{env}}(\rho_{\text{total}}) \quad (2.15)$$

where

$$\text{Tr}_{\text{env}}(\rho_{\text{total}}) \doteq \sum_{m \in \text{env}} (\mathbb{I} \otimes \langle m|) \rho_{\text{total}} (\mathbb{I} \otimes |m\rangle) \quad (2.16)$$

In the end we will obtain a quantum state ρ_G composed by n qubits that encapsulates the information of the original graph G by effectively incorporating the entanglement effects introduced during the encoding process. Although the final representation needs only n qubits, a quantum computer with $2n$ qubits will be needed to perform the encoding.

2.2 VARIATIONAL QUANTUM GRAPH CONVOLUTION

During this step parameterized quantum circuits perform non-linear graph convolutions, incorporating higher-order neighborhood interac-

tions to model transaction flows evolving through the layers with the non linear operator

$$\rho^{(0)} = \rho_G \quad (2.17)$$

$$\rho^{(l+1)} = \mathcal{N}_\theta^{(l)}(\mathcal{U}_\theta^{(l)} \rho^{(l)} \mathcal{U}_\theta^{(l)\dagger}) \quad (2.18)$$

where $\mathcal{N}_\theta^{(l)}(\rho)$ is the non-linear quantum channel defined as

$$\mathcal{N}_\theta^{(l)}(\rho) = \sum_k p_k(\theta) \Pi_k \rho \Pi_k \quad (2.19)$$

$\mathcal{U}_\theta^{(l)}$ is the unitary operator. The original paper defines it as

$$\mathcal{U}_\theta^{(l)} = \exp \left(-i \sum_{(i,j) \in E} \theta_{ij}^{(l)} H_{ij} - i \sum_{i \in V} \phi_i^{(l)} \sigma_i^z - i \sum_{(i,j,k) \in \Delta} \psi_{ijk}^{(l)} \sigma_i^z \sigma_j^z \sigma_k^z \right) \quad (2.20)$$

This definition is subject to the same dimensionality flaw already identified for the Hamiltonian $H(G)$. Summing σ_i^z over all vertices V and $\sigma_i^z \sigma_j^z \sigma_k^z$ over all hyperedges Δ implicitly assumes N distinct qubits, which is incompatible with the n -qubit amplitude-encoding space.

Following the same reasoning used for the Hamiltonian, my solution is to redefine the variational unitary using the effective Hamiltonian H_{eff} [5], now parameterized per layer:

$$\mathcal{U}_\theta^{(l)} = \exp(-i H_{\text{eff}}(\Theta^{(l)})) \quad (2.21)$$

where the per-layer effective Hamiltonian is

$$H_{\text{eff}}(\Theta^{(l)}) = \sum_{q=0}^{n-1} \phi_q^{(l)} \sigma_q^z + \sum_{q < r}^{n-1} \theta_{qr}^{(l)} \sigma_q^z \sigma_r^z + \sum_{q < r < s}^{n-1} \psi_{qrs}^{(l)} \sigma_q^z \sigma_r^z \sigma_s^z \quad (2.22)$$

This substitution makes the VQC mathematically executable on NISQ hardware. However, as with the Hamiltonian correction, it comes at a cost: the model no longer performs explicit topological message passing in the hidden layers. The VQC effectively acts as a non-linear phase classifier, and the structural information of the graph must be fully carried by the initial state ρ_G produced during the GAES encoding step.

In the original paper the Kraus operators Π_k and the probabilities $p_k(\theta)$ are not explicitly defined. In my implementation, in regard to preserve the

node information (diagonal terms of the density matrix) while filtering the complex relational patterns, i chose to implement the *Phase-Damping (Dephasing) Channel*. This should break unitarity, enabling the model to learn complex, non-linear boundaries between legitimate and fraudulent transactions [8]. The mathematical definition of Kraus operators becomes

$$\Pi_1 = I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \Pi_2 = \sigma^z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad (2.23)$$

Notice that they both satisfy the completeness relation

$$\sum_k E_k^\dagger E_k = I \quad (2.24)$$

The associated probabilities depend on the tunable parameters θ_1, θ_2 and are defined as

$$p_0(\theta) = \frac{e^{\theta_0}}{e^{\theta_0} + e^{\theta_1}}, \quad p_1(\theta) = \frac{e^{\theta_1}}{e^{\theta_0} + e^{\theta_1}} \quad (2.25)$$

The bigger will be the value of θ_1 the more noise will be added to the system. θ should be initialized in a way that $p_0(\theta) \gg p_1(\theta)$, the network will learn to increase the "noise" (phase damping) in deeper layers to strip away irrelevant correlations and focus on the core topological features of the fraud graph.

$\theta_{ij}^{(l)}$, $\phi_i^{(l)}$ and $\psi_{ijk}^{(l)}$ are tunable parameters that captures transaction dynamics, account-specific dynamics and multi-party (hyperedge) interactions respectively. The quantum feature embedding after L layers will be

$$Z_Q = \text{Tr}_{\text{anc}} \left(\prod_{l=1}^L \mathcal{N}_\theta^{(l)} (U_\theta^{(l)} \rho_G U_\theta^{(l)\dagger}) \right) \quad (2.26)$$

Where Tr_{anc} is defined as for Tr_{env} and it traces out ancilla qubits that were introduced to apply the Kraus operators.

We also compute the correlation entropy which captures multi-scale transaction interactions

$$C_Q = \sum_{i \neq j} \text{Tr}(\rho_{ij}^{(L)} \log \rho_{ij}^{(L)} - \rho_i^{(L)} \log \rho_i^{(L)} - \rho_j^{(L)} \log \rho_j^{(L)}) \quad (2.27)$$

where $\text{Tr}(\rho) = \sum_k \rho_{kk}$.

Both Z_Q and C_Q will be used as features in the final classification.

2.3 TOPOLOGICAL QUANTUM SIGNATURE EXTRACTION

In this section we will extract Quantum Betti numbers and persistent Euler characteristic from the quantum-encoded graph.

Definition 1 (Betti numbers). *Betti numbers act as a "counter" for different types of holes in a topological space or object. Each Betti number β_i corresponds to the number of i -cells in the CW-complex (graph). In particular, β_0 is the number of connected components, β_1 counts the number of one-dimensional holes (Cycles), β_2 counts the number of two-dimensional voids (cavities). In our methods higher-order Quantum Betti numbers are computed to extend the analysis beyond simple loops.*

Definition 2 (Euler's characteristic). *Let τ be a CW-complex composed by β_i i -cells. We define Euler's characteristic as*

$$\chi(\tau) = \sum_{i=0}^{\infty} (-1)^i \beta_i \quad (2.28)$$

The Euler's characteristic is a topological invariant that combines information from all Betti numbers into a single scalar value.

The original paper doesn't specify how many Betti numbers are computed, in my implementation I decided to compute up to β_3 to capture complex topological features.

From now on we define $\rho \doteq \rho^{(L)}$ from the last layer of the Variational Quantum Graph Convolution. Persistent homology is extended to compute higher-order topological invariants over a multi-scale quantum distance landscape, capturing structural anomalies in transaction networks. The quantum distance matrix uses a weighted metric defined as follows.

$$d_\rho(i, j) = \sqrt{1 - F_w(\rho_i, \rho_j)} \quad (2.29)$$

where

$$F_w(\rho_i, \rho_j) = \frac{(\text{Tr} \sqrt{\sqrt{\rho_i} W \sqrt{\rho_j}})^2}{\text{Tr}(W)} \quad (2.30)$$

and W is a positive definite weight matrix encoding priors like transaction frequency and account centrality. The quantum Vietoris–Rips complex is defined as the simplicial complex formed by connecting nodes that are within a distance ϵ of each other.

$$\mathcal{VR}_\epsilon(\rho) = \{\sigma \subseteq V \mid d_\rho(i, j) \leq \epsilon, \forall i, j \in \sigma\} \quad (2.31)$$

Then the Quantum Betti numbers $\beta_k(\epsilon)$ are computed as

$$\beta_k^Q(\epsilon) = \dim H_k(\mathcal{VR}_\epsilon(\rho); \mathbb{Q}) \quad (2.32)$$

where H_k is the k -th homology group with rational coefficients defined as

$$H_k(\mathcal{VR}_\epsilon(\rho); \mathbb{Q}) = \frac{\ker \partial_k}{\text{im } \partial_{k+1}} \quad (2.33)$$

with $\partial_k : C_k \rightarrow C_{k-1}$ is the boundary operator, a linear map that takes a k -dimensional shape and returns its $(k-1)$ -dimensional boundary.

Once the Quantum Betti numbers are computed we can define the Persistent Euler Characteristic (PEC) as

$$\chi^Q(\epsilon) = \sum_k (-1)^k \beta_k^Q(\epsilon) \quad (2.34)$$

The value of ϵ should be tuned and, since $d_\rho(i, j) \in [0, \sqrt{2}]$, we should choose values in that range. More than one value can be chosen and used as features for the final classification. Adding those topological features to the quantum features Z_Q and C_Q we obtain the final feature vector

$$\phi(G) = [Z_Q, C_Q, \{\beta_k^Q(\epsilon)\}_{k, \epsilon}, \{\chi^Q(\epsilon)\}_\epsilon] \quad (2.35)$$

2.4 QUANTUM-CLASSICAL HYBRID ANOMALY LEARNING

In this section a hybrid quantum-classical architecture is proposed to perform the final anomaly detection.

PROBLEM: THE DEFINITION OF S_{NORMAL} In the original paper a weight vector S_{normal} is required as input and it's described as a representation of normal transaction patterns. The exact definition was not provided but from the definition of the loss function it's clear that S_{normal} should contain as values the feature vectors $\phi(G)$ of graphs that are known to be legitimate. This, from my prospective, is a big flaw in the original paper. In fact, the value of $\phi(G)$, $G \in \text{legitimate}$ cannot be known before the training and therefore it cannot be passed as input to the model. I'm sure that in their implementation they found a way to compute an estimate of S_{normal} and maybe then update it during the training, but I think that it should have been mentioned in the paper.

To overcome this issue i firstly tried a method inspired by "Deep One-Class Classification" [9] that collapses normal data into a single mean vector (see Appendix A). Unfortunately, the assumption of using a hypersphere to model normal transactions turned out to be too restrictive and the model struggled to generalize to unseen fraud patterns.

By implementing S_{normal} as a Memory Bank we maintain a dynamic collection of feature vectors from the most recent legitimate graphs. This allows the model to represent the "normal" class as a multimodal manifold, effectively capturing distinct clusters of valid behavior (e.g., high-frequency small trading vs. low-frequency large transfers) without averaging them into an inaccurate center.

We define the Memory Bank at training step t as a set $\mathcal{M}^{(t)}$ with fixed capacity K :

$$\mathcal{M}^{(t)} \doteq \{\mathbf{m}_1, \mathbf{m}_2, \dots, \mathbf{m}_K\} \quad (2.36)$$

where each $\mathbf{m}_k \in \mathbb{R}^d$ is a stored feature vector $\phi(G)$ of a previously processed legitimate graph. The bank is updated via a First-In-First-Out (FIFO) queue mechanism. Let $V_{\text{normal}}^{(t)}$ be the set of feature vectors from legitimate graphs in the current batch.

To ensure the unsupervised loss is well-defined at the start of training, the memory bank is initialized using a "warm-up" phase. Let $\mathcal{G}_{\text{init}} \subset \mathcal{G}_{\text{normal}}$ be a subset of K legitimate graphs sampled before training begins. We define the initial state $\mathcal{M}^{(0)}$ as:

$$\mathcal{M}^{(0)} \doteq \{\phi(G; \Theta^{(0)}) \mid G \in \mathcal{G}_{\text{init}}\} \quad (2.37)$$

where $\Theta^{(0)}$ represents the randomly initialized network parameters. This ensures that the bank contains a representative distribution of the untrained feature space, preventing model collapse during the early training iterations. Afterwards the update rule is:

$$\mathcal{M}^{(t+1)} \leftarrow \left(\mathcal{M}^{(t)} \setminus \mathcal{M}_{\text{oldest}} \right) \cup V_{\text{normal}}^{(t)} \quad (2.38)$$

where $\mathcal{M}_{\text{oldest}}$ represents the set of vectors removed to make space for the new entries.

The anomaly detection is formulated as a hypothesis test based on the distance to the *nearest* neighbor in the memory bank, rather than a single center:

$$\mathcal{H}_0 : \min_{\mathbf{m} \in \mathcal{M}} \|\phi(G) - \mathbf{m}\|^2 \leq \tau \quad (\text{Normal}) \quad (2.39)$$

$$\mathcal{H}_1 : \min_{\mathbf{m} \in \mathcal{M}} \|\phi(G) - \mathbf{m}\|^2 > \tau \quad (\text{Anomaly}) \quad (2.40)$$

The loss function is defined as

$$\mathcal{L} = \mathcal{L}_{\text{sup}}(\phi(G), y; \Theta) + \lambda_1 \mathcal{L}_{\text{unsup}}(\phi(G), \mathcal{M}) + \lambda_2 \mathcal{R}(\Theta) \quad (2.41)$$

where the supervised term remains:

$$\mathcal{L}_{\text{sup}}(\phi(G), y; \Theta) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \quad (2.42)$$

with $\hat{y} = \sigma(W^\top \phi(G) + b)$. The unsupervised loss term is redefined to minimize the distance between a normal graph's embedding and its closest prototype in the bank:

$$\mathcal{L}_{\text{unsup}}(\phi(G), \mathcal{M}) = \min_{\mathbf{m} \in \mathcal{M}} \|\phi(G) - \mathbf{m}\|^2 \quad (2.43)$$

$$\mathcal{R}(\Theta) = \sum_l \|\Theta^{(l)}\|_2^2 \quad (2.44)$$

λ_1, λ_2 are hyperparameters controlling the trade-off between supervision, manifold consistency, and regularization. Θ represent the adaptive parameters optimized via quantum gradient flows.

$$\Theta^{(t+1)} = \Theta^{(t)} - \eta \nabla_{\Theta} \mathcal{L} \left(\mathcal{N} \left(e^{-iH_{\text{eff}}(\Theta)} \rho_{\text{in}} e^{iH_{\text{eff}}(\Theta)} \right) \right) \quad (2.45)$$

This should lead to a robust learning of normal and fraudulent transaction patterns across quantum and classical domains.

After the training we define S_{normal} using a validation set W composed by M non fraudulent graphs as

$$S_{\text{normal}} = \{\phi(G^*) \mid G^* \in W\} \quad (2.46)$$

S_{normal} will not be modified during the inference phase.

2.5 DECISION AND INTERPRETABILITY

The final decision is made based on the distance from the learned normal pattern S_{normal} .

$$s(G) = \min_{\phi(G^*) \in S_{\text{normal}}} \|\phi(G) - \phi(G^*)\|_2^2 \quad (2.47)$$

The decision rule is

$$\text{Fraud}(G) \doteq \begin{cases} 1 & \text{if } s(G) > \tau \\ 0 & \text{otherwise} \end{cases} \quad (2.48)$$

Where τ is a threshold that can be tuned based on the desired trade-off between false positives and false negatives. My implementation is slightly different from the one described in the original paper since I used a different definition for S_{normal} .

A PREVIOUS IMPLEMENTATION OF S_{normal}

My first implementation of S_{normal} turned out to be ineffective; the assumption of using an hypersphere to model normal transactions was too restrictive and the model struggled to generalize to unseen fraud patterns. I mention it here for completeness.

The idea was to compute S_{normal} as the mean of the feature vectors of all the legitimate graphs in the current batch. This technique is inspired by the paper *“Deep One-Class Classification”* [10]. This changes the network the network to map all "normal" data points into a hypersphere of minimum volume surrounding a center c . By doing so the model should learn common factors of variation in normal transactions, making it better at detecting "unknown" types of fraud that fall far from that center.

The new definition of S_{normal} becomes

$$S_{\text{normal}}^{(0)} = \mu_i \doteq \frac{1}{|\mathcal{G}_{\text{normal}}^{(i)}|} \sum_{G \in \mathcal{G}_{\text{normal}}^{(i)}} \phi(G) \quad (\text{A.1})$$

$$S_{\text{normal}}^{(i)} = \alpha \cdot S_{\text{normal}}^{(i-1)} + (1 - \alpha) \cdot \mu_i \quad (\text{A.2})$$

where $\mathcal{G}_{\text{normal}}^{(i)}$ is the set of legitimate graphs in the i -th batch and $\alpha \in [0, 1]$ is a hyperparameter. If I would have followed the implementation described in the paper *“Deep One-Class Classification”* [10] S_{normal} would have been computed as the mean of the normal sample in the batch; our implementation compute it as a moving average over batches as this idea is proven to improve robustness [12].

The anomaly detection is still formulated as an hypothesis test

$$\mathcal{H}_0 : \|\phi(G) - S_{\text{normal}}^{(i)}\|^2 \leq R^2 \quad (\text{Normal}) \quad (\text{A.3})$$

$$\mathcal{H}_1 : \|\phi(G) - S_{\text{normal}}^{(i)}\|^2 > R^2 \quad (\text{Anomaly}) \quad (\text{A.4})$$

The loss function is defined as

$$\mathcal{L} = \mathcal{L}_{\text{sup}}(\phi(G), y; \Theta) + \lambda_1 \mathcal{L}_{\text{unsup}}(\phi(G), \Theta) + \lambda_2 \mathcal{R}(\Theta) \quad (\text{A.5})$$

where

$$\mathcal{L}_{\text{sup}}(\phi(G), y; \Theta) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \quad (\text{A.6})$$

where $\hat{y} = \sigma(W^\top \phi(G) + b)$ is the predicted probability of being an anomaly, with σ being the sigmoid function, W and b are learnable parameters, and $y \in \{0, 1\}$ is the ground truth label (0 for normal, 1 for anomaly).

$$\mathcal{L}_{\text{unsup}}(\phi(G), \Theta) = \|\phi(G) - S_{\text{normal}}^{(i)}\|^2 \quad (\text{A.7})$$

$$\mathcal{R}(\Theta) = \sum_l \|\Theta^{(l)}\|_2^2 \quad (\text{A.8})$$

λ_1, λ_2 are hyperparameters that control the relative importance of the unsupervised, and regularization terms respectively. Θ are adaptive parameters optimized via quantum gradient flows

BIBLIOGRAPHY

- [1] Alfredo Contreras and Edgar Villa Pérez. Strategic choices in money laundering: Smurfing, layering, and financial over-diversification. *Journal of Economic Criminology*, 11:100199, 2026. (Cited on pages 10 and 11.)
- [2] Mohammad Doost and Mohammad Manthouri. Quantum topological graph neural networks for detecting complex fraud patterns. *arXiv preprint arXiv:2512.03696*, 2025. (Cited on pages 1, 11, 12, and 14.)
- [3] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. 2014. (Cited on page 15.)
- [4] Bryce Goodman and Seth Flaxman. European union regulations on algorithmic decision-making and a "right to explanation". *AI magazine*, 38(3):50–57, 2017. (Cited on page 1.)
- [5] Zhirui Hu, Jinyang Li, Zhenyu Pan, Shanglin Zhou, Lei Yang, Caiwen Ding, Omer Khan, Tong Geng, and Weiwen Jiang. On the design of quantum graph convolutional neural network in the nisq-era and beyond. In *2022 IEEE 40th International Conference on Computer Design (ICCD)*, pages 290–297, 2022. (Cited on page 17.)
- [6] Xiaoqi Li, Wenkai Li, Zhijie Liu, Meikang Qiu, Zhiquan Liu, Sen Nie, Zongwei Li, Shi Wu, and Yuqing Zhang. Scamsweeper: Detecting illegal accounts in web3 scams via transactions analysis. *IEEE Transactions on Information Forensics and Security*, 2025. (Cited on page 10.)
- [7] P. Monkam, F. De Lucia, and M. Bastian. Topological data analysis in financial networks. *Journal of Computational Finance*, 2024. (Cited on page 11.)
- [8] MarÃa Laura Olivera-Atencio, Lucas Lamata, and JesÃs Casado-Pascual. Impact of amplitude and phase damping noise on quantum reinforcement learning: Challenges and opportunities. 2025. (Cited on page 18.)

- [9] T. Pourhabibi, K.-L. Ong, B. H. Kam, and Y. L. Boo. Fraud detection: A systematic literature review of graph-based anomaly detection approaches. *Decision Support Systems*, 133:113303, 2020. (Cited on page 13.)
- [10] Lukas Ruff, Robert Vandermeulen, Nico Goernitz, Lucas Deecke, Shoaib Ahmed Siddiqui, Alexander Binder, Emmanuel Muller, and Marius Kloft. Deep one-class classification. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4393–4402. PMLR, 10-15 Jul 2018. (Cited on page 25.)
- [11] E. Umeda, K. Kaneko, and M. Kikuchi. Quantum machine learning for financial data. *Quantum Information Processing*, 2019. (Cited on page 12.)
- [12] Xingbao Zhang, Wei Li, and Yue Zhao. One-class anomaly detection with redundancy reduction and momentum mechanism. pages 1–6, 2022. (Cited on page 25.)